

# Pubmed RAG

Diseño e implementación de un sistema RAG biomédico sobre PubMed y  
PubMed Central

FastAPI · LangChain · Qdrant · bge-m3 · OpenAI / Gemini / Ollama · Docker

Gloria del Río  
2026

# PARTE I

## Construcción del sistema RAG

*El núcleo del proyecto: el pipeline RAG, sus decisiones de diseño y las alternativas descartadas.*

### 1. Resumen ejecutivo

El proyecto consiste en el diseño e implementación de un sistema RAG biomédico completo: un pipeline que clasifica la intención de una consulta en lenguaje natural, recupera evidencia de un corpus de PubMed y PubMed Central con la estrategia adecuada a esa intención, y genera una respuesta fundamentada exclusivamente en los documentos recuperados, citando cada fuente. Ese pipeline — clasificación, retrieval adaptativo, construcción de contexto, generación — constituye el sistema. Todo lo demás que el proyecto incorpora — múltiples proveedores de LLM, un frontend de demostración, actualización incremental del corpus, migración de la integración con NCBI — son funcionalidades y mejoras de ingeniería construidas sobre ese núcleo, no el núcleo en sí.

El sistema combina una base de datos vectorial con filtrado nativo sobre metadatos (Qdrant), un modelo de embeddings biomédico seleccionado mediante evaluación comparativa (bge-m3) y un LLM que genera respuestas citadas. Se expone a través de una API REST en FastAPI, con una interfaz web de demostración, y está completamente contenerizado con Docker. Cada decisión de diseño del pipeline — qué base de datos vectorial, qué modelo de embeddings, qué framework de API, qué librería de orquestación — se tomó frente a alternativas concretas evaluadas y descartadas, no por defecto.

### 2. Contexto y motivación

La literatura biomédica crece de forma continua — PubMed indexa más de 40 millones de registros —, lo que dificulta localizar y sintetizar evidencia relevante para una pregunta concreta. Un LLM generalista puede generar respuestas plausibles sobre un tema médico, pero no garantiza que estén fundamentadas en evidencia científica concreta ni permite verificar de dónde procede cada afirmación.

Un sistema RAG reduce este problema recuperando primero documentos relevantes de un corpus verificado y limitando la generación al contexto obtenido, en lugar de apoyarse en el conocimiento paramétrico del modelo. Esto exige, además de la recuperación semántica, trazabilidad de las citas y un mecanismo para mantener el corpus actualizado a medida que se publican y revisan artículos — sin lo cual el sistema degradaría con el tiempo igual que cualquier índice estático.

### 3. Objetivos

#### 3.1 Objetivo general

Diseñar e implementar un sistema RAG biomédico capaz de consultar, ampliar y actualizar un corpus científico procedente de PubMed y PMC (PubMed Central).

#### 3.2 Objetivos específicos

1. Integrar metadatos de PubMed y textos completos de PMC.
2. Generar embeddings biomédicos localmente, sin dependencia de un proveedor cloud.

3. Almacenar y recuperar fragmentos en Qdrant con filtrado por metadatos.
4. Clasificar automáticamente el tipo de consulta antes de recuperar información.
5. Adaptar la estrategia de retrieval a cada tipo de pregunta.
6. Generar respuestas con fuentes citadas y verificables.
7. Permitir la ingesta manual de nuevos artículos bajo demanda.
8. Implementar actualizaciones incrementales del corpus ya indexado.
9. Evitar que las actualizaciones bloqueen el resto de la API.
10. Permitir distintos proveedores de LLM sin modificar el pipeline RAG.
11. Proporcionar una interfaz web para consultar, ingestar y actualizar el corpus.
12. Contenerizar la solución completa con Docker.
13. Validar el sistema mediante pruebas automatizadas y funcionales.

## 4. Alcance y limitaciones

### 4.1 Alcance

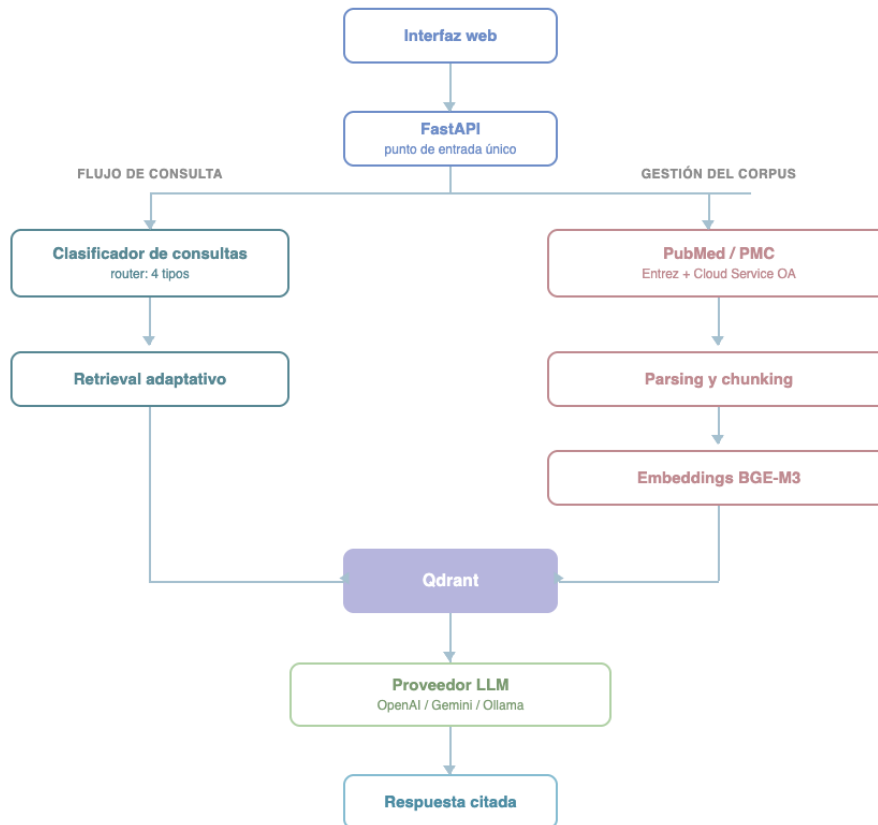
El sistema permite:

1. consultas temáticas sobre un conjunto de artículos;
2. consultas sobre un artículo concreto;
3. consultas transversales entre varias publicaciones;
4. ingesta temática de nuevos artículos;
5. actualización incremental del corpus ya indexado;
6. consulta del estado del corpus;
7. selección de proveedor LLM por configuración;
8. visualización de las fuentes utilizadas en cada respuesta.

### 4.2 Fuera de alcance

1. diagnóstico médico;
2. recomendación clínica individualizada;
3. evaluación automática de la calidad metodológica de cada estudio;
4. garantía de cobertura completa de toda la literatura biomédica;
5. despliegue productivo multiusuario;
6. evaluación clínica de las respuestas generadas.

## 5. Arquitectura general



Interfaz web y API como punto de entrada único, bifurcado en flujo de consulta y gestión del corpus, convergiendo en Qdrant y en el proveedor LLM activo.

La interfaz web y la API forman el punto de entrada único. A partir de ahí el sistema se bifurca en dos flujos independientes: el flujo de consulta — el pipeline RAG propiamente dicho, desarrollado bloque a bloque en la sección siguiente — y la gestión del corpus — ingesta bajo demanda y actualización incremental, tratadas en la Parte II como evolución del sistema. Ambos flujos convergen en la misma colección de Qdrant y en el mismo proveedor LLM. Esta separación permite razonar sobre cada flujo de forma independiente: la gestión del corpus nunca bloquea una consulta, y una consulta nunca modifica el corpus.

## 6. El pipeline RAG, bloque a bloque

Esta es la sección que define el proyecto. El resto de la memoria explica cómo se construyó y cómo evolucionó el sistema; esta sección explica cómo piensa: qué ocurre, en qué orden y por qué, entre que un usuario escribe una pregunta y recibe una respuesta citada.

Usuario → Clasificador → Tipo de consulta → Retrieval adaptativo → Qdrant → Contexto → Prompt → LLM → Respuesta

Este recorrido revela una idea que atraviesa el resto de la memoria: en un sistema RAG, la calidad de la respuesta no la determina el modelo de lenguaje, sino la cadena que lo precede — los datos, el chunking, los embeddings, los metadatos y el retrieval. El LLM interviene al final, sobre un contexto que ya ha decidido, en gran medida, si la respuesta puede llegar a ser correcta.

### 6.1 Clasificador: qué evidencia necesita cada pregunta

Antes de recuperar nada, un LLM clasifica la pregunta del usuario en uno de cuatro tipos — thematic\_summary, specific\_query, transversal\_query, none — y extrae identificadores

cuando están presentes (autor, título de paper, PMC ID). Esta clasificación decide tanto el filtro que se aplicará sobre el payload de Qdrant (source=pubmed o source=pmc) como la técnica de recuperación que usará el bloque siguiente.

El aprendizaje subyacente es que no existe una estrategia de recuperación universal: el tipo de evidencia que necesita una consulta condiciona el retrieval desde el primer paso del pipeline, antes incluso de tocar Qdrant.

| Tipo de consulta     | Qué necesita el lector                                | Estrategia de retrieval                                                                                          |
|----------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Resumen temático     | Visión general sobre un tema                          | Filtra por source=pubmed y recupera los 6 abstracts más relevantes                                               |
| Paper concreto       | Detalle de un estudio específico                      | Resuelve el pmc_id (autor → título → fallback semántico) y recupera los 10 chunks más relevantes de ese artículo |
| Consulta transversal | Mecanismo o hallazgo compartido entre varios estudios | Filtra por source=pmc y aplica MMR para maximizar diversidad entre papers                                        |
| Fuera de dominio     | Pregunta ajena al corpus biomédico                    | Sin retrieval: responde explicando qué puede resolver el sistema                                                 |

Decisión: clasificar antes de recuperar, no después. Se descartó la alternativa de ejecutar siempre una búsqueda semántica amplia sobre toda la colección y filtrar o reordenar los resultados a posteriori. Esa estrategia aplicaría el mismo procedimiento a una pregunta que necesita seis abstracts diversos y a una pregunta que necesita los diez fragmentos de un único artículo, y el volumen de contexto resultante sería con frecuencia inadecuado para el caso concreto. **Clasificar primero** permite fijar el número de documentos, el filtro y la **técnica de búsqueda** correctos desde el primer paso, en lugar de corregirlos después de una recuperación genérica.

## Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ● [u](#)

### Consultar

Ingestar

Actualizar

#### Pregunta

¿Qué evidencia existe sobre la relación entre microbiota intestinal y depresión?

Ejemplos:

Resumen temático

Artículo concreto

Comparación transversal

Fuera de dominio

### Preguntar

CONSULTA  
"¿Qué evidencia existe sobre la relación entre microbiota intestinal y depresión?"

CLASIFICACIÓN  
Resumen temático  
► Detalle

RETRIEVAL  
Abstracts de PubMed  
► Detalle

MODELO  
OpenAI · GPT-4.1-mini

Interfaz "Consultar": clasificación, retrieval y modelo activo visibles antes de la respuesta, con las seis fuentes PubMed citadas al final.

## 6.2 Retrieval adaptativo: una estrategia por tipo de consulta

El retrieval no ejecuta la misma consulta a Qdrant en los tres casos.

Para un artículo concreto, el sistema resuelve primero el pmc\_id en tres pasos con degradación progresiva: filtro por apellido de autor si la pregunta lo menciona, búsqueda semántica con el título extraído por el clasificador, y como último recurso búsqueda semántica con la pregunta completa. Si ninguno de los tres resuelve el identificador, la consulta degrada automáticamente a transversal en lugar de fallar — prioriza devolver una respuesta útil, aunque menos precisa, frente a bloquear al usuario por un fallo de resolución.

### Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ●

**Consultar** Ingestar Actualizar

#### Pregunta

¿Qué metodología utilizaron en el estudio PMC13341303?

Ejemplos: [Resumen temático](#) [Artículo concreto](#) [Comparación transversal](#) [Fuera de dominio](#)

#### PMC ID

PMC12345678 o 12345678

Opcional: indica el PMC ID si quieres apuntar directamente a un artículo. Si lo dejas vacío, el sistema intentará identificarlo a partir de tu pregunta (autor, título...).

**Preguntar**

CONSULTA "¿Qué metodología utilizaron en el estudio PMC13341303?" → CLASIFICACIÓN Artículo concreto ▶ Detalle → RETRIEVAL Texto completo — PMC13341303 ▶ Detalle → MODELO OpenAI · GPT-4.1-mini

#### RESULTADO

La metodología utilizada en el estudio PMC13341303 consistió en un diseño longitudinal, auto-controlado, pre-post, donde se evaluaron pacientes con diabetes tipo 2 antes y después de una intervención de ejercicio Yijinjing durante 6 meses [4]. Se recolectaron muestras fecales para análisis del microbioma intestinal mediante secuenciación del gen 16S rRNA en la región V3, utilizando la plataforma Illumina MiSeq, y se agruparon secuencias con >97% de similitud en unidades taxonómicas operativas (OTUs) para evaluar la riqueza y diversidad microbiana [3]. Se midieron variables antropométricas como peso, altura, índice de masa corporal y composición corporal mediante bioimpedancia [6]. Se realizaron análisis bioquímicos y de marcadores inmunológicos en sangre, incluyendo glucosa en ayunas, insulina, lípidos, hemoglobina glucosilada, citoquinas inflamatorias (TNF- $\alpha$ , IL-6, IL-1 $\beta$ , IL-10, CRP) y marcadores de permeabilidad intestinal (D-lactato, Zonulina, MFG-E8) mediante ensayos ELISA [5]. Para el análisis estadístico se usó SPSS 19.0, aplicando ANOVA para datos normales y pruebas no paramétricas para datos no normales, con análisis de correlación de Pearson o Spearman según correspondiera, y análisis de correlación parcial controlando por cambios en composición corporal [1]. La intervención de Yijinjing consistió en sesiones de 45-60 minutos, 5 veces por semana, con supervisión y entrenamiento técnico inicial para asegurar la adherencia y correcta ejecución [9]. El protocolo contó con aprobación ética y consentimiento informado de los participantes [4].

#### FUENTES UTILIZADAS (10)

TEXTO COMPLETO · PMC  
Metabolic regulatory mechanisms of Yijinjing exercise in patients with type 2 diabetes mellitus: Insight from the gut microbiota-intestinal barrier- inflammation axis  
PMC13341303 · Statistical analysis  
[Mostrar fragmento](#)

TEXTO COMPLETO · PMC  
Metabolic regulatory mechanisms of Yijinjing exercise in patients with type 2 diabetes mellitus: Insight from the gut microbiota-intestinal barrier- inflammation axis  
PMC13341303 · Limitations and future directions  
[Mostrar fragmento](#)

*El campo PMC ID opcional permite apuntar directamente al artículo; si se deja vacío, el sistema lo infiere de la propia pregunta siguiendo los tres pasos de resolución.*

Para una consulta transversal, el retrieval aplica **MMR (Maximal Marginal Relevance)** en lugar de una búsqueda por similitud pura. Se descartó la búsqueda por similitud simple porque, con un k moderado, tiende a devolver varios chunks del mismo artículo — el más cercano al vector de la consulta — en lugar de cubrir distintos papers. MMR penaliza la redundancia entre resultados ya seleccionados, lo que favorece la cobertura de múltiples artículos y secciones a costa de una relevancia marginalmente menor en cada resultado individual: exactamente el **trade-off que necesita una pregunta que busca un mecanismo compartido entre estudios**.

## Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ● 

Consultar Ingestar Actualizar

### Pregunta

¿Qué mecanismos conectan los probióticos con la mejora del síndrome metabólico según los distintos estudios?

Ejemplos:

 Resumen temático

 Artículo concreto

 Comparación transversal

 Fuera de dominio

### Preguntar

#### CONSULTA

"¿Qué mecanismos conectan los probióticos con la mejora del síndrome metabólico según los distintos estudios?"

#### CLASIFICACIÓN

Comparación transversal

→ ▶ Detalle

#### RETRIEVAL

Texto completo, varios artículos (PMC)

→ ▶ Detalle

#### MODELO

OpenAI · GPT-4.1-mini

#### RESULTADO

Los probióticos mejoran el síndrome metabólico a través de varios mecanismos interrelacionados según los estudios revisados:

1. **\*\*Modulación del microbioma intestinal\*\***: Los probióticos aumentan la diversidad del microbioma, especialmente reintroduciendo cepas beneficiosas como *Lactobacillus* y *Bifidobacterium*, lo que conduce a cambios metabólicos y hormonales favorables, incluyendo la reducción de niveles de testosterona y mejora de la resistencia a la insulina [1].
2. **\*\*Mejora del metabolismo de ácidos biliares\*\***: La modulación del microbioma influye positivamente en el metabolismo de ácidos biliares, que a su vez regula el metabolismo de glucosa y lípidos a través de receptores FXR y TGR5 [1].
3. **\*\*Producción de ácidos grasos de cadena corta (SCFA)\*\***: Los probióticos aumentan la producción de SCFA, que fortalecen la barrera mucosa intestinal, previenen la endotoxemia y combaten la resistencia a la insulina [1][4].
4. **\*\*Reducción de la inflamación sistémica\*\***: Los probióticos disminuyen moléculas proinflamatorias como TNF-α e IL-6, reduciendo la inflamación crónica asociada al síndrome metabólico y mejorando la función hepática y muscular [1][4][5].
5. **\*\*Restauración de la integridad de la barrera intestinal\*\***: Al mejorar la función de la barrera intestinal, los probióticos reducen la permeabilidad intestinal y la translocación de endotoxinas (como LPS), lo que disminuye la activación inflamatoria sistémica [4].
6. **\*\*Sinergia con compuestos bioactivos vegetales\*\***: La combinación de probióticos con fitoquímicos puede potenciar efectos antimicrobianos, cooperación metabólica, modulación inmune y estabilización ecológica del microbioma, reforzando la barrera intestinal y mejorando la salud metabólica [3].

En conjunto, estos mecanismos contribuyen a mejorar la sensibilidad a la insulina, reducir la inflamación crónica, preservar la función hepática y muscular, y estabilizar el metabolismo glucolipídico, lo que favorece la mejora del síndrome metabólico [1][3][4][5].

*La respuesta enumera los mecanismos identificados citando, por afirmación, qué artículo la respalda — varias afirmaciones citan más de una fuente cuando coinciden.*

## FUENTES UTILIZADAS (6)

TEXTO COMPLETO · PMC

### Dietary interventions for modulating the gut microbiome in PCOS management

PMC12957162 · Probiotic supplementation

Mostrar fragmento

TEXTO COMPLETO · PMC

### Metabolic regulatory mechanisms of Yijinjing exercise in patients with type 2 diabetes mellitus: Insight from the gut microbiota-intestinal barrier- inflammation axis

PMC13341303 · Correlation analysis between gut microbiota, inflammatory cytokines, and intestinal barrier markers

Mostrar fragmento

TEXTO COMPLETO · PMC

### Probiotic-Plant Bioactive Synergy in Gut Health: Mechanisms, Antimicrobial Activity, and Translational Challenges

PMC13363466 · 5.7. Conceptual Model of Probiotic-Phytochemical Synergy

Mostrar fragmento

TEXTO COMPLETO · PMC

### Decoding the mechanistic basis of liver-muscle communication in health and disease

PMC13269367 · Gut microbiota: implications for liver fibrosis and muscle atrophy

Mostrar fragmento

TEXTO COMPLETO · PMC

### Impact of continuous probiotic supplementation on intestinal barrier function and hepatic biomarkers in fulminant liver failure models

PMC12963265 · Cytokine profiling (ELISA)

Mostrar fragmento

TEXTO COMPLETO · PMC

### The interplay of dietary sugar, chronic inflammation, and bladder cancer: mechanistic insights, evidence, and prevention strategies

PMC12926174 · Dietary patterns, glycaemic control, and microbiome modulation in bladder cancer risk

Mostrar fragmento

Cada fuente citada puede expandirse para mostrar el fragmento exacto que fundamenta la respuesta — la trazabilidad llega hasta el texto, no solo hasta el título del artículo.


**National Library of Medicine**  
National Center for Biotechnology Information

[Search](#)
[Log in](#)


**PubMed Central®**

[Journal List](#)
[User Guide](#)

As a library, NLM provides access to scientific literature. Inclusion in an NLM database does not imply endorsement of, or agreement with, the contents by NLM or the National Institutes of Health. Learn more: [PMC Disclaimer](#) | [PMC Copyright Notice](#)




► [Nutrients](#). 2026 Jun 28;18(13):2112. doi: [10.3390/nu18132112](#)

### Probiotic-Plant Bioactive Synergy in Gut Health: Mechanisms, Antimicrobial Activity, and Translational Challenges

[Monika Elzbieta Jach](#)<sup>1,\*</sup>, [Ewa Sajnaga](#)<sup>2</sup>, [Ewa Ozimek](#)<sup>3</sup>, [Anna Seresko](#)<sup>4</sup>, [Marcello Locatelli](#)<sup>5</sup>

Editors: Lina Yang, Jihua Chen

► [Author information](#) ► [Article notes](#) ► [Copyright and License information](#)

PMCID: PMC13363466 PMID: [42451116](#)

#### Abstract

**Background/Objectives:** Antimicrobial resistance (AMR), microbiota disruption, and chronic inflammation have intensified the search for alternative and complementary antimicrobial strategies. Probiotics and plant-derived bioactive compounds (phytochemicals) are increasingly being investigated as microbiota-supporting, immunomodulatory, and antimicrobial agents. This review synthesizes the current evidence on probiotic-phytochemical interactions, with particular emphasis on mechanisms relevant to antimicrobial synergy, gut barrier reinforcement, microbiota modulation, and translational development. **Methods:** A narrative literature review with a structured search strategy was conducted using major scientific databases, including PubMed, Scopus, EBSCO, Google Scholar, SpringerLink, Wiley Online Library, and Taylor & Francis, and open repositories.

#### ACTIONS


[View on publisher site](#)


[PDF \(2.3 MB\)](#)


[Cite](#)


[Collections](#)


[Permalink](#)

#### RESOURCES

[Similar articles](#)
+

[Cited by other articles](#)
+

[Links to NCBI Databases](#)
+

#### ON THIS PAGE

[Abstract](#)

- [1. Introduction](#)
- [2. Methodology of the Review—Narrative Literature Review with a Structured Search Strategy](#)
- [3. Mechanisms of the Antibacterial Action of Probiotic Microorganisms](#)
- [4. Antibacterial and Immunomodulatory Activity of Plant Extracts](#)
- [5. Interactions Between Probiotics and Phytochemicals: From Complementarity to Synergy](#)
- [6. Methodological Approaches to Evidence](#)


[Feedback](#)

Cada PMCID citado por el sistema corresponde a un artículo real y verificable en [pmc.ncbi.nlm.nih.gov](https://pmc.ncbi.nlm.nih.gov) — la trazabilidad de la respuesta llega hasta la fuente pública original.

## Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ●

Consultar Ingestar Actualizar

### Pregunta

¿Qué tratamiento debería seguir para mi enfermedad?

Ejemplos: Resumen temático Artículo concreto Comparación transversal Fuera de dominio

### Preguntar

CONSULTA "¿Qué tratamiento debería seguir para mi enfermedad?" → CLASIFICACIÓN Fuera de dominio → RETRIEVAL Sin retrieval → MODELO OpenAI · GPT-4.1-mini

### RESULTADO

Lo siento, pero solo puedo ayudar con información basada en artículos científicos biomédicos indexados, resúmenes de evidencia a nivel de tema, preguntas sobre metodología, resultados y conclusiones de artículos específicos, y búsquedas de evidencia entre artículos dentro del corpus indexado. Para recomendaciones personalizadas sobre tratamientos, te recomiendo consultar directamente con un profesional de la salud. ¿Puedo ayudarte con información científica sobre alguna enfermedad o tratamiento específico?

Guardrail: El router marca "sin retrieval", no se malgasta una consulta a Qdrant en una pregunta que ya se sabe fuera de dominio.

## 6.3 De los chunks al contexto

Los documentos que devuelve Qdrant no se pasan al LLM tal cual: se formatean con cabeceras numeradas [1], [2]... antes de insertarse en el prompt. Esa numeración es la misma que el LLM reutiliza al citar y la misma que ve el usuario en la lista de fuentes de la respuesta — no existe una traducción intermedia entre el orden de recuperación y la cita final. Esta decisión evita una clase entera de errores: si el número de cita se calculase por separado de la lista de fuentes mostrada, un desajuste entre ambos generaría citas que apuntan a la fuente equivocada sin que el sistema pueda detectarlo.

## 6.4 Prompt y generación: el grounding

El prompt de generación fija principios que no dependen del proveedor LLM activo: usar únicamente el contexto recuperado, nunca inventar datos, indicar explícitamente cuando la evidencia disponible es insuficiente, citar cada afirmación por el número de fuente correspondiente, responder en español y separar con claridad el contexto recuperado de la respuesta generada. En un dominio donde una afirmación clínica incorrecta tiene coste, este mecanismo de grounding limita el espacio en el que el modelo puede alucinar: la respuesta no puede contener una afirmación que no esté respaldada por alguno de los fragmentos numerados del contexto.

Qué modelo de lenguaje concreto ejecuta este prompt (OpenAI, Gemini u Ollama) resulta transparente para el resto del pipeline. La Parte II (sección 12) describe la implementación de una factoría de proveedores LLM, basada en el patrón Factory, que desacopla la lógica del sistema del proveedor de inferencia utilizado. De este modo, el mecanismo de grounding descrito en esta sección se mantiene inalterado con independencia del modelo configurado.

El grounding cierra la cadena Datos → Chunking → Embeddings → Metadatos → Retrieval → Contexto → Grounding → LLM planteada al inicio de esta sección: el LLM es, en la práctica, el último eslabón, y su margen de maniobra depende de todo lo que ocurrió antes. En un dominio biomédico esta idea deja de ser solo arquitectónica y pasa a ser un requisito del propio dominio: recuperar correctamente la evidencia resulta más importante que maximizar la fluidez de la respuesta, y la trazabilidad deja de ser una funcionalidad adicional para convertirse en una condición del sistema.

## 7. Fuentes de datos

El sistema combina dos fuentes de datos complementarias, ambas accesibles a través del sistema Entrez de NCBI. PubMed actúa como fuente primaria de descubrimiento y proporciona metadatos bibliográficos: título, resumen, autores, fecha e identificadores. PubMed Central (PMC) actúa como fuente secundaria de enriquecimiento y proporciona texto completo estructurado por secciones, junto con la licencia de reutilización, cuando el artículo está disponible en acceso abierto.

PubMed ofrece una cobertura bibliográfica más amplia — incluye artículos sin texto completo accesible —, mientras que PMC permite realizar recuperación sobre el texto completo de los artículos disponibles en acceso abierto. Por eso la ingesta trata ambas fuentes como capas: PubMed siempre aporta el registro base, y PMC lo enriquece cuando existe y su licencia lo permite (véase sección 16, Licencias).

## 8. Procesamiento documental y chunking

Cada artículo atraviesa una secuencia fija de pasos entre su recuperación y su disponibilidad para el retrieval: resolución y validación del identificador, extracción de metadatos bibliográficos, comprobación de licencia, descarga y parseo del XML de PMC cuando aplica, separación por secciones, limpieza del contenido, división en chunks cuando la sección lo requiere, generación de embeddings e inserción en Qdrant.

Cada punto insertado conserva un conjunto de metadatos — título, identificador PMC, fuente, sección, licencia e identificador del artículo — junto al texto vectorizado. Esta redundancia deliberada permite filtrar por metadatos sin una segunda consulta, y citar y explicar el origen de cada fragmento en la respuesta final sin volver a tocar Qdrant.

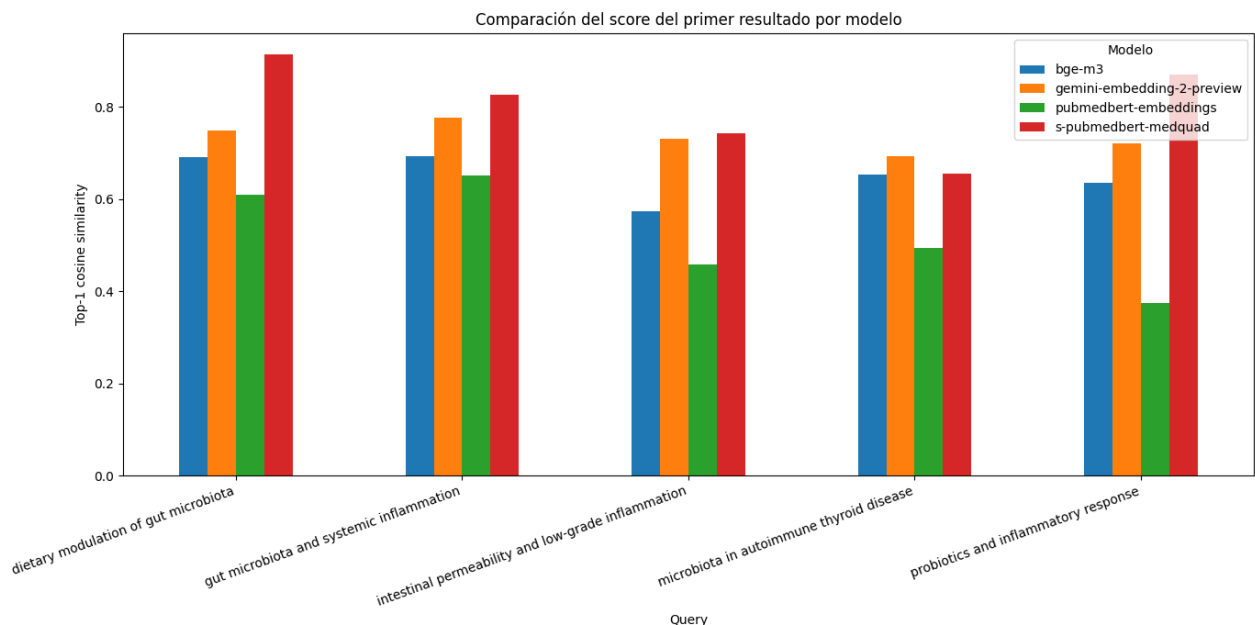
El criterio de chunking es adaptativo, no fijo: las secciones por debajo de un umbral de tokens se indexan como unidad completa, preservando su coherencia semántica; las que lo superan se subdividen en fragmentos con solapamiento, para no perder continuidad semántica entre fragmentos contiguos de una misma sección. Se descartó un tamaño de chunk fijo para todo el corpus porque las secciones de un artículo científico varían mucho en longitud según su función — una sección de métodos detallada puede ser varias veces más larga que una de conclusiones — y un tamaño único penalizaría a un extremo de esa distribución: fragmentos demasiado grandes pierden capacidad discriminativa en el embedding, fragmentos demasiado pequeños fragmentan artificialmente secciones que ya eran coherentes.

## 9. Embeddings

Se utiliza bge-m3, ejecutado localmente mediante sentence-transformers, sobre CPU. La elección responde a varios factores combinados: independencia del proveedor LLM activo, sin coste variable por token embebido; funcionamiento local, sin llamadas de red en el camino crítico del retrieval; soporte multilingüe; y una representación semántica que resultó más robusta que las alternativas evaluadas.

Decisión descartada: modelos biomédicos especializados. Se evaluaron dos alternativas de dominio — pubmedbert-base-embeddings y S-PubMedBert-MedQuAD — frente a bge-m3 (generalista).

| Modelo                     | Dim. | Tiempo corpus (s) | Tiempo queries (s) | Comportamiento observado                                               |
|----------------------------|------|-------------------|--------------------|------------------------------------------------------------------------|
| bge-m3                     | 1024 | 87,34             | 0,82               | Estable y consistente, bajo ruido semántico                            |
| pubmedbert-base-embeddings | 768  | 14,80             | 0,12               | Resultados correctos pero poco discriminativos                         |
| S-PubMedBert-MedQuAD       | 768  | 12,15             | 0,057              | Puntuación alta en casos puntuales, mayor variabilidad entre consultas |



Score de similitud del primer resultado recuperado, por modelo y consulta de prueba. El benchmark exploratorio incluyó además un modelo de embeddings cloud (gemini-embedding-2-preview), no llevado a la comparación detallada de la tabla anterior.

Los **modelos biomédicos** son de tres a seis veces **más rápidos**, pero muestran mayor variabilidad: en varias consultas de prueba, S-PubMedBert-MedQuAD recuperó documentos

alineados semánticamente con la temática de la consulta pero fuera de foco clínico, por ejemplo, artículos sobre sepsis o estabilidad genómica en consultas centradas en inflamación intestinal, introduciendo ruido que bge-m3 no producía. En un sistema de retrieval, un documento relevante que queda fuera del espacio vectorial próximo a la consulta no llega nunca al LLM, independientemente de la calidad del modelo generativo: por eso **se priorizó la consistencia de bge-m3 frente al mejor desempeño puntual de los modelos especializados**.

Un modelo con comportamiento predecible reduce el riesgo de **fallos silenciosos de recall**, aunque implique mayor coste computacional — y ese coste, ejecutado en CPU local sin llamadas de red, no compite en la práctica con la latencia de red de un proveedor de embeddings cloud.

La experiencia mostró que maximizar la similitud del embedding no implica necesariamente mejorar la recuperación útil para el usuario: un modelo puede puntuar mejor un resultado y, al mismo tiempo, ser menos fiable en el conjunto de las consultas. La métrica que importa no es la puntuación de un caso concreto, sino la consistencia con la que el sistema recupera evidencia relevante en el rango completo de preguntas que puede recibir.

## 10. Base de datos vectorial: Qdrant

Toda la colección vive en un único índice (biomedical\_rag), diferenciando puntos de PubMed y de PMC por el campo source del payload. El esquema de datos incorpora metadatos densos por documento — fuente, fecha, idioma, licencia, PMCID — que en un sistema RAG son la base del filtrado que acota el espacio de búsqueda antes o durante la recuperación vectorial. Ese filtrado, no la escalabilidad ni la latencia, fue el criterio decisivo en la elección de base de datos.

| Criterio                  | Relevancia | Motivo                                                                   |
|---------------------------|------------|--------------------------------------------------------------------------|
| Filtrado sobre metadatos  | Crítico    | Determina la calidad del retrieval en los cuatro casos de uso del router |
| Persistencia              | Alta       | Los embeddings deben sobrevivir entre sesiones                           |
| Integración con LangChain | Alta       | Simplifica la construcción del pipeline RAG                              |
| Infraestructura local     | Alta       | El despliegue no depende de servicios cloud                              |
| Latencia                  | Media      | El corpus es acotado; no es el factor limitante                          |
| Escalabilidad horizontal  | Baja       | Proyecto demostrativo con corpus reducido                                |

Otras bases de datos contempladas:

1. **Weaviate**: filtrado por post-procesamiento sobre el índice vectorial, en lugar de durante el recorrido del índice HNSW como hace Qdrant; menos eficiente en el escenario de filtrado intensivo que necesita este sistema.
2. **Pinecone**: sin despliegue local posible, un servicio SaaS de pago incompatible con el requisito de infraestructura local del proyecto.

3. **Milvus**: mayor capacidad de escalabilidad horizontal, pero exige una infraestructura basada en Kubernetes desproporcionada para el volumen de este corpus; sería la opción natural si el sistema escalara al corpus completo de PubMed, no para un proyecto demostrativo.
4. **FAISS**: librería de búsqueda vectorial eficiente, pero carece de persistencia nativa, API de servicio y soporte de metadatos estructurados; no resuelve el problema de filtrado que motiva esta decisión, ni constituye una base de datos como tal.
5. **pgvector**: válida cuando ya existe una base de datos relacional en el sistema, pero la búsqueda vectorial no es su función principal y su rendimiento en tareas ANN de alta dimensionalidad es inferior al de soluciones especializadas; este proyecto no parte de una base de datos relacional previa que justificara la extensión.

Qdrant aplica el filtrado sobre metadatos durante el recorrido del **índice HNSW** en lugar de como post-procesamiento, gestiona de forma nativa los vectores de 1024 dimensiones que produce bge-m3, permite despliegue local mediante Docker y se integra directamente con LangChain. Es la única alternativa evaluada que cumple simultáneamente los **tres requisitos** críticos del proyecto: filtrado eficiente por metadatos, despliegue sin infraestructura cloud, e integración directa con el resto del pipeline.

## 11. API

| Método | Endpoint              | Propósito                                                                        |
|--------|-----------------------|----------------------------------------------------------------------------------|
| GET    | /health               | Estado de Qdrant, embeddings y proveedor LLM, verificados de forma independiente |
| POST   | /query                | Responde una pregunta en lenguaje natural con fuentes citadas                    |
| POST   | /ingest               | Amplía el corpus con una nueva consulta temática (síncrono)                      |
| GET    | /corpus/status        | Recuento de artículos, chunks y licencias indexadas                              |
| POST   | /corpus/update        | Lanza la actualización incremental del corpus (asíncrono, con job_id)            |
| GET    | /corpus/update/status | Estado del job de actualización en curso, para hacer polling                     |

Ejemplo de respuesta de /query:

```
{
  "question": "...", "answer": "...", "query_type": "thematic_summary",
  "reason": "...", "retrieval_strategy": "...", "pmc_id": null,
  "sources": [{"title": "...", "section": null, "pmc_id": "PMC...",
    "pm_id": "...", "source": "pubmed", "excerpt": "..."}]
}
```

### 11.1 Por qué FastAPI

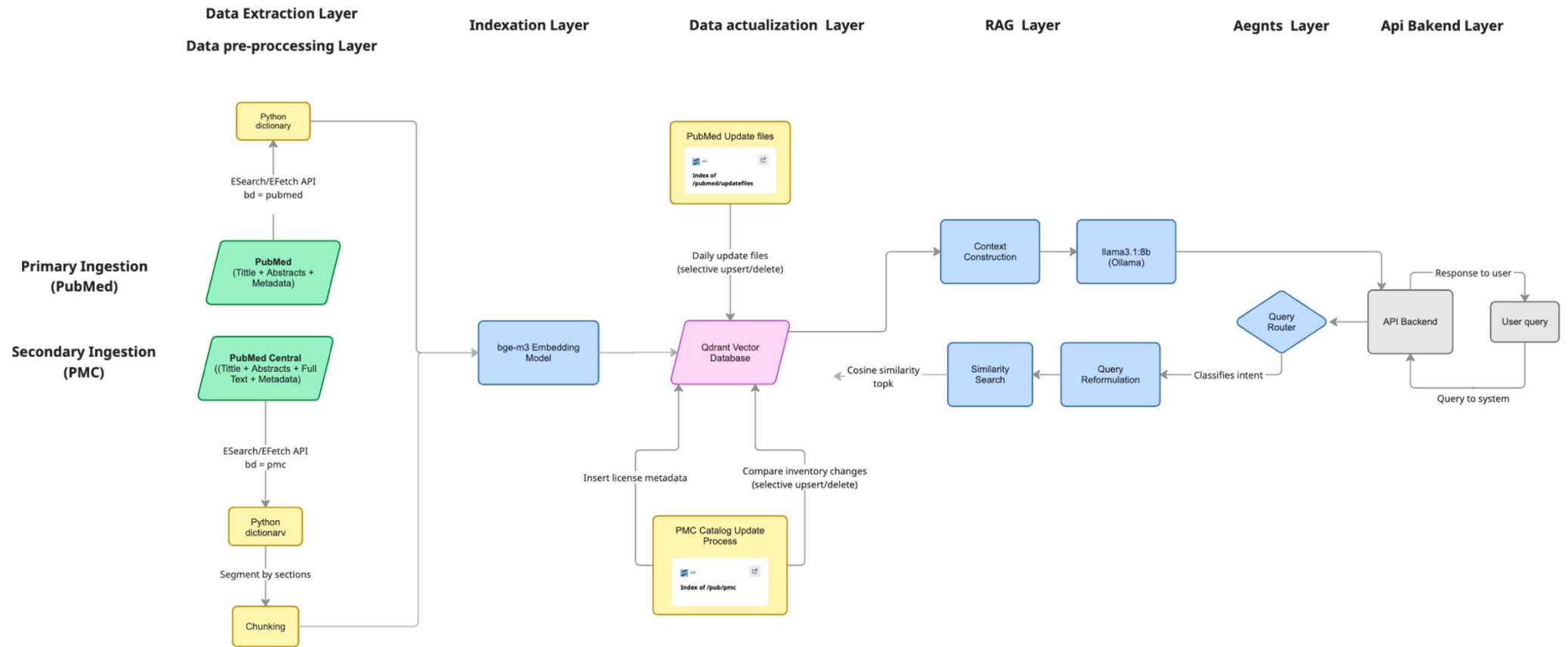
Los modelos de request y response del sistema — QueryRequest, IngestResponse, UpdateJobStatusResponse — se definen una única vez como clases Pydantic y sirven a la vez

de documentación interactiva (/docs) y de contrato validado en tiempo de ejecución: una petición con un campo mal formado se rechaza con 422 antes de llegar al pipeline RAG, sin código de validación manual. El soporte async nativo es relevante para las llamadas de red hacia NCBI sin bloquear el resto de la API.

Frente a Flask, que exigiría añadir Pydantic y la especificación OpenAPI como piezas independientes y mantenerlas sincronizadas a mano, o Django REST Framework, pensado para aplicaciones con modelo de datos relacional y panel de administración que este proyecto no necesita, FastAPI cubre exactamente la superficie que requiere una API REST delgada sobre un pipeline RAG: validación, documentación y async en una sola pieza.

## 12. LangChain

Orquestar el pipeline implica encadenar pasos heterogéneos — prompt de clasificación con salida estructurada, prompt de generación, tres proveedores de LLM intercambiables — que LangChain expone con una interfaz común (Runnable) y un operador de composición que hace explícita la cadena clasificador → retrieval → generación directamente en el código. La alternativa de construir estas cadenas a mano habría significado reescribir, por separado para OpenAI, Gemini y Ollama, la lógica de invocar cada proveedor y parsear su salida. LangChain ya resuelve esa capa de abstracción, y es precisamente lo que permite que la arquitectura multiproveedor (sección 12) se reduzca a una función de fábrica en lugar de tres implementaciones paralelas del pipeline completo.



Extracción desde PubMed y PMC, chunking adaptativo y generación de embeddings antes de llegar a Qdrant.

## PARTE II

### Evolución del sistema

*Cómo evolucionó la arquitectura de la Parte I al enfrentarse a restricciones reales de uso: límites de un único proveedor, mantenimiento del corpus en el tiempo, infraestructura de despliegue.*

## 13. Arquitectura multiproveedor LLM

El primer punto en el que la arquitectura de la Parte I tuvo que evolucionar fue, precisamente, el eslabón que la memoria sitúa como el menos determinante para la calidad de la respuesta: el proveedor de LLM. El proveedor que ejecuta el clasificador y la generación (sección 6) se selecciona mediante un patrón de fábrica: un registro asocia cada valor de LLM\_PROVIDER (openai, gemini u ollama) con una función constructora que valida únicamente las variables de ese proveedor. El resultado es un modelo de chat de LangChain que el resto del pipeline consume de forma genérica, sin conocer el proveedor concreto detrás — por eso es una mejora aditiva sobre el pipeline de la Parte I, no un cambio en cómo razona el sistema.

La configuración se resuelve una única vez al importar el módulo, en el arranque del proceso; cambiar de proveedor en Docker exige recrear el contenedor con la nueva variable ya presente en el archivo de entorno. Aporta portabilidad frente a un único proveedor, control de costes, tolerancia a límites de cuota, experimentación con modelos y despliegue completamente local con Ollama cuando no se quiere depender de ningún servicio externo.

## 14. Migración de la integración con NCBI

### 14.1 Situación inicial

El diseño original resolvía el estado de acceso abierto de PMC descargando `oa_file_list.csv`, un catálogo global de índice completo publicado en el antiguo FTP Service de PMC. Cada actualización del corpus comparaba la versión descargada con una copia local para detectar altas, bajas y cambios de licencia en todo el catálogo, no solo en los artículos ya indexados por este sistema.

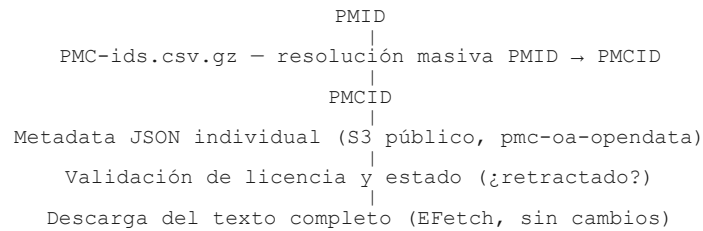
### 14.2 Problema

NCBI retiró `oa_file_list.csv` del FTP Service en 2026, con retirada definitiva anunciada para agosto de 2026. El flujo anterior dejó de ser sostenible: depender de un fichero deprecado y con fecha de desaparición conocida no era una base razonable para el módulo de actualización incremental.

### 14.3 Investigación realizada

El anuncio oficial de NCBI describía los recursos que sustituyen al FTP Service. Se identificaron dos recursos con responsabilidades distintas: `PMC-ids.csv.gz` (un fichero masivo pero no deprecado, útil únicamente para resolver PMID → PMCID) y los recursos del PMC Cloud Service: un bucket S3 público (`pmc-oa-opensdata`) que expone metadatos de acceso abierto — licencia, estado de retractación, última modificación — en un JSON individual por artículo, accesible por HTTPS sin credenciales.

### 14.4 Nueva arquitectura



El punto de diseño central es separar tres responsabilidades que el flujo anterior confluía en un único catálogo: resolución de identificadores (masiva, un fichero para todo PMC), metadatos de licencia y retractación (por artículo, consultados solo para lo que el sistema realmente indexa) y descarga de texto completo (sin cambios, vía E-Utilities). Un fallo de red al consultar la metadata de un artículo se trata como “indeterminado, dejar como está”, nunca como señal de baja — solo un 404 limpio confirma que un artículo ya no está en el subconjunto de acceso abierto.

## 14.5 Mejora obtenida

El mantenimiento del corpus ya no necesita descargar ni diferenciar un catálogo global de varios cientos de megabytes en cada ciclo de actualización: solo consulta, artículo a artículo, los `pmc_id` que ya están indexados. Esto reduce la transferencia y el almacenamiento necesarios en cada actualización, permite consultas selectivas en lugar de un barrido completo, y adapta el proyecto a la infraestructura vigente de NCBI en lugar de depender de un servicio con fecha de caducidad anunciada.

## 15. Actualización asíncrona del corpus

### 15.1 Problema inicial

La actualización del corpus es una operación costosa, ya que implica consultar los servicios de NCBI, descargar y procesar documentos XML, generar nuevos chunks y calcular sus correspondientes embeddings antes de incorporarlos a la base de datos vectorial. Durante este proceso se realizan tanto operaciones de entrada/salida (peticiones HTTP) como tareas intensivas de procesamiento en CPU.

Si todo este trabajo se ejecutara dentro del mismo proceso que atiende las peticiones de FastAPI, la API dejaría de responder con normalidad mientras dura la actualización. Endpoints como `/health` o `/query` quedarían bloqueados o verían degradado su tiempo de respuesta hasta finalizar el proceso.

Aunque parte del trabajo puede ejecutarse en un hilo independiente, las fases de procesamiento —como el parseo de XML, el chunking y la generación de embeddings— son tareas intensivas de CPU. En CPython, este tipo de operaciones están limitadas por el Global Interpreter Lock (GIL), por lo que utilizar únicamente hilos no evita que compitan por el tiempo de ejecución del intérprete.

### 15.2 Solución

Para evitar que la actualización del corpus afecte a la disponibilidad de la API, el proceso se ejecuta mediante un `ProcessPoolExecutor` con un único worker. A diferencia de un hilo, el trabajo se realiza en un proceso independiente del sistema operativo, lo que permite aislar las tareas intensivas de CPU del proceso principal de FastAPI.

Al ejecutarse en un proceso independiente, el worker no comparte memoria con la API. Por este motivo, el estado de la actualización (`idle`, `running`, `completed` o `failed`) se persiste en un

fichero JSON en disco, de forma que pueda ser consultado tanto por la API como por el frontend durante el polling. Si el contenedor se reinicia mientras una actualización está en curso, FastAPI detecta un estado running que ya no tiene un proceso asociado y lo marca explícitamente como failed, evitando que el frontend espere indefinidamente un trabajo que nunca finalizará.

Como el proyecto no incorpora autenticación, fue necesario introducir un mecanismo sencillo para **evitar un uso abusivo del endpoint de actualización**. Para ello, el inicio de cada trabajo se protege mediante un lock, que garantiza que solo pueda arrancarse una actualización simultáneamente, y un **periodo de cooldown** de cinco minutos desde la última ejecución. Con ello se evita que múltiples peticiones consecutivas desencadenen actualizaciones repetidas sobre los servicios de NCBI y la base de datos vectorial.

```

POST /corpus/update
  |
  +-- Lock + cooldown (409 si hay job activo, 429 si < 5 min)
  |
  +-- Job en ProcessPoolExecutor (proceso independiente)
  |
  +-- Estado persistido en JSON (idle / running / completed / failed)
  |
  +-- GET /corpus/update/status <- polling cada 3s desde el frontend

```

## Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ●

Consultar

Ingestar

**Actualizar**

### ESTADO DEL CORPUS

|           |           |            |
|-----------|-----------|------------|
| PUBMED    | PMC       | CHUNKS     |
| <b>30</b> | <b>19</b> | <b>499</b> |

LICENCIAS ⓘ

CC BY

**19**

Última actualización: 20/7/26, 14:08

Esta operación consulta PubMed y PubMed Central para detectar artículos nuevos, modificados o retirados entre los ya indexados, y actualiza el corpus. No amplía el corpus — eso es lo que hace la pestaña Ingestar. Puede tardar varios minutos.

**Actualizar corpus**

La pestaña Actualizar muestra el estado actual del corpus antes de lanzar el job: consulta PubMed y PMC para detectar cambios en lo ya indexado.

## 16. Interfaz web

El último paso de esta evolución fue exponer el pipeline y la gestión del corpus sin exigir tocar la API directamente. El frontend es estático — HTML, CSS y módulos ES de JavaScript sin build step ni framework —, servido directamente por FastAPI y montado como catch-all después de todas las rutas de la API. La decisión evita añadir un segundo proceso de build a un proyecto de portfolio: el navegador ejecuta los módulos tal cual, y el volumen de Docker que monta el directorio estático como solo lectura permite iterar el frontend sin reconstruir la imagen.

### 16.1 Consultar

La pestaña que expone el pipeline de la Parte I: pregunta libre, cuatro ejemplos precargados y visualización del pipeline completo — consulta, clasificación, retrieval, modelo — con detalle

técnico expandible en cada paso. Las capturas de esta pestaña ya se mostraron en la sección 6, junto al bloque del pipeline que ilustran.

Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ●

Consultar
Ingestar
Actualizar

Pregunta

Escribe una pregunta sobre el corpus biomédico indexado...

Ejemplos:
Resumen temático
Artículo concreto
Comparación transversal
Fuera de dominio

Preguntar

## 16.2 Ingestar

Query temática y número de artículos; deshabilita el botón de envío mientras la petición está en curso como guarda simple contra el doble envío. Renderiza un checklist con el resultado — artículos encontrados, indexados en PubMed, encontrados y excluidos por licencia en PMC, XML descargados, chunks creados — y un log ampliable.

Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ●

Consultar
Ingestar
Actualizar

La ingesta puede tardar varios minutos (búsqueda en PubMed, cruce con PMC, chunking e indexación). No cierres esta pestaña mientras esté en curso.

Query temática

gut microbiota & intestinal permeability

Número de artículos

10

Ingestar

- ✓ 10 artículos encontrados
- ✓ 10 PubMed indexados
- ✓ 8 PMC encontrados
- ✓ 1 excluidos por licencia restrictiva
- ✓ 7 XML descargados
- ✓ 212 chunks creados
- ✓ Qdrant actualizado

► Ver log

*Diez artículos encontrados, uno excluido por licencia restrictiva — el checklist hace visible cada paso del pipeline de ingesta, incluida la exclusión por licencia.*

## 16.3 Actualizar

Estado del corpus (artículos PubMed, artículos PMC, chunks, desglose de licencias) y un botón de actualización con modal de confirmación. Al aceptar, hace polling sobre el estado del job cada tres segundos, mostrando la fase activa. Si se recarga la página o se cambia de pestaña mientras un job sigue en curso, la pestaña lo detecta al activarse y retoma el polling en lugar de asumir que está inactivo.

## Biomedical RAG

API ● Qdrant ● Embeddings: BGE-M3 ● LLM: OpenAI ●

Consultar

Ingestar

**Actualizar**

### ESTADO DEL CORPUS

| PUBMED    | PMC       | CHUNKS     |
|-----------|-----------|------------|
| <b>30</b> | <b>19</b> | <b>499</b> |

LICENCIAS ⓘ

CC BY

19

Última actualización: 20/7/26, 14:08

Esta operación consulta PubMed y PubMed Central para detectar artículos nuevos, modificados o retirados entre los ya indexados, y actualiza el corpus. No amplía el corpus — eso es lo que hace la pestaña Ingestar. Puede tardar varios minutos.

### Actualizar corpus

- ✓ PubMed revisados: 30
- ✓ Artículos PubMed modificados: 0
- ✓ PMC revisados: 19
- ✓ PMC reindexados: 6
- ✓ Sin errores

▶ Ver log

*Resultado de una actualización incremental del corpus. La interfaz muestra el estado actual de la colección y un resumen de los cambios detectados durante el proceso.*

## 17. Licencias

El conjunto de licencias permitidas está definido en `ALLOWED_PMC_LICENSES` (`src/config.py`) y se aplica en dos puntos del sistema: durante la ingesta (`src/ingest/pipeline.py`), para decidir si se descarga e indexa el texto completo, y durante el mantenimiento del corpus (`src/data_actualization/oa_updater.py`), para retirar el texto completo de los artículos cuya licencia haya cambiado.

Se indexa el texto completo únicamente cuando la licencia permite su reutilización y redistribución:

1. **CC BY** — atribución.
2. **CC BY-SA** — atribución y obligación de compartir las obras derivadas bajo la misma licencia.
3. **CC BY-NC** — atribución y uso no comercial.
4. **CC BY-NC-SA** — uso no comercial y obligación de compartir las obras derivadas bajo la misma licencia.
5. **CCO** — dominio público.

Por el contrario, se excluyen deliberadamente las licencias ND (No Derivatives) — **CC BY-ND** y **CC BY-NC-ND** —, ya que no permiten distribuir obras derivadas: un sistema RAG genera respuestas construidas a partir de fragmentos de múltiples documentos, por lo que resulta más prudente no indexar contenido sujeto a esta restricción. También se excluye cualquier licencia o código no Creative Commons, cuyas condiciones de reutilización pueden ser incompatibles o no permitir la redistribución del contenido.

Gracias a este filtrado, el sistema garantiza que únicamente incorpora al corpus textos completos cuya licencia es compatible con el uso previsto, manteniendo además la trazabilidad de la licencia asociada a cada documento.

## 18. Contenerización

Dos servicios en docker-compose: qdrant (imagen oficial, volumen persistente para el almacenamiento vectorial) y api (construida desde el Dockerfile del proyecto, puerto 8000). El contenedor de la API monta tres volúmenes con propósitos distintos: uno persistente para los ficheros de estado de la ingesta y la actualización, uno de solo lectura para el frontend estático, y un volumen nombrado para la caché de Hugging Face, que evita volver a descargar bge-m3 (~1,1 GB) en cada rebuild.

El Dockerfile instala explícitamente la variante CPU-only de torch antes del resto de dependencias: sentence-transformers arrastra torch de forma transitiva, y el wheel por defecto de PyPI en Linux instala el runtime completo de CUDA (~2 GB de librerías nvidia-\*), aunque el proyecto ejecuta los embeddings exclusivamente en CPU. Instalar primero el wheel CPU-only del índice oficial de PyTorch hace que la instalación posterior del resto de dependencias encuentre torch ya satisfecho y nunca resuelva la variante CUDA.

Para usar Ollama en local, el contenedor de la API alcanza el Ollama del host a través de host.docker.internal, ya que Ollama no forma parte de este docker-compose — se ejecuta aparte, en el host, con o sin GPU disponible.

## PARTE III

### Validación

*Pruebas, resultados y conclusiones.*

## 19. Pruebas y validación

Once archivos de test cubren riesgos distintos, no una única capa de la aplicación:

| Área                      | Qué se valida                                                                                                   | Archivo                      |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------|
| Factoría LLM              | Selección de proveedor sin llamadas reales; error claro si falta configuración; independencia entre proveedores | test_llm_factory.py          |
| Proveedor activo          | Sanity check real: la clave configurada funciona con una llamada mínima                                         | test_openai_api_available.py |
| Salud del sistema         | Qdrant, embeddings y LLM desacoplados; degradación parcial cuando Qdrant cae                                    | test_health.py               |
| Ingesta                   | Pipeline completo end-to-end sobre una query real predefinida                                                   | test_indexing.py             |
| Retrieval semántico       | Los tres patrones de búsqueda directamente sobre Qdrant, sin API ni LLM                                         | test_semantic_search.py      |
| API end-to-end            | Cinco casos — los cuatro tipos de consulta más un caso fuera de dominio — contra la API completa                | test_api.py                  |
| Actualización incremental | Pipeline PubMed y PMC sobre una colección de Qdrant de prueba, aislada de producción, sin red real              | test_data_actualization.py   |
| Cliente PMC OA            | Resolución PMID → PMCID y metadata de licencia, totalmente mockeado                                             | test_pmc_oa_client.py        |
| Router y retrieval        | Clasificación de scope y limpieza de contenido, con Qdrant mockeado                                             | test_routing_retrieval.py    |
| Job de actualización      | Estado del job, endpoints vía TestClient (conurrencia, cooldown) y un ProcessPoolExecutor real                  | test_update_job.py           |
| Actualización (CLI)       | Pipeline PubMed/PMC con flags --pubmed-only, --pmc-only y --dry-run                                             | test_actualizations.py       |

La validación funcional completó la cobertura automática con verificación manual sobre el stack real: ingesta real, actualización real, las tres consultas del router más el caso fuera de dominio, visualización de fuentes y comprobación del frontend completo en navegador.

## 20. Aprendizajes durante el diseño del pipeline RAG

La construcción del pipeline descrito en la Parte I puso de manifiesto un conjunto de conclusiones sobre el comportamiento de un sistema RAG que no eran evidentes antes de construirlo. Algunas ya se apuntaron, de forma puntual, al cierre de las secciones donde surgieron; esta sección las recoge todas juntas y las conecta entre sí, porque es en conjunto — no aisladas — donde muestran la idea que atraviesa el proyecto: la calidad de un sistema RAG se decide antes de llegar al LLM. Estas conclusiones, más que los problemas de infraestructura recogidos en la sección siguiente, son las que explican por qué el sistema se diseñó como se diseñó.

### 20.1 Los embeddings biomédicos no siempre mejoran la recuperación

Se esperaba que los modelos entrenados específicamente sobre literatura biomédica ofrecieran mejores resultados que un modelo generalista. Las pruebas comparativas mostraron, sin embargo, que aunque estos modelos alcanzaban puntuaciones de similitud superiores en determinadas consultas, también introducían mayor variabilidad y recuperaban documentos menos alineados con la intención real de la pregunta (sección 9). Por ello se priorizó la consistencia de bge-m3 frente al mejor rendimiento puntual de los modelos especializados: en un sistema de retrieval, un fallo silencioso de recall resulta más costoso que una puntuación subóptima en un caso concreto.

### 20.2 El retrieval condiciona la calidad de la respuesta más que el proveedor de LLM

Independientemente de la capacidad del modelo generativo, una respuesta solo puede ser tan buena como el contexto que recibe: si el retrieval no recupera el fragmento relevante, ningún LLM puede fundamentar la respuesta en él. Esta relación de dependencia sitúa el diseño del retrieval — no la elección del proveedor de LLM — como el factor que determina la calidad final del sistema, y explica por qué la arquitectura multiproveedor (sección 12) pudo tratarse como una mejora de ingeniería aditiva: cambiar de LLM no compensa un retrieval mal diseñado.

### 20.3 No todas las preguntas necesitan la misma estrategia de retrieval

El router de clasificación (secciones 6.1 y 6.2) surgió de esta constatación: aplicar siempre la misma búsqueda semántica sobre la colección completa no ofrece un rendimiento uniforme entre tipos de pregunta. Una consulta que necesita una visión general se satisface con un conjunto reducido de abstracts diversos; una consulta sobre un artículo concreto necesita profundidad sobre un único documento; una consulta transversal necesita cobertura entre varios documentos evitando redundancia. Tratar estos tres casos con la misma técnica de recuperación degradaría el resultado en al menos dos de los tres.

### 20.4 Abstracts y texto completo cumplen funciones distintas en el retrieval

La distinción entre PubMed y PMC (sección 7) no es solo una diferencia de fuente de datos, sino de función dentro del pipeline. Los abstracts de PubMed son apropiados para preguntas de visión general porque condensan el contenido de un artículo en un espacio semántico

reducido y homogéneo; el texto completo de PMC es necesario para preguntas sobre metodología, resultados o detalles específicos, que no están representados en un resumen. Un sistema que solo indexara abstracts no podría responder con precisión a la segunda categoría de preguntas; uno que solo indexara texto completo perdería eficiencia y cobertura en la primera.

## 20.5 La calidad de los metadatos es tan determinante como la calidad del embedding

Un embedding preciso no es suficiente si el punto que lo acompaña no conserva la información necesaria para filtrar, citar y verificar la fuente. Campos como el identificador PMC, la sección de origen, la fuente (PubMed o PMC) y la licencia (secciones 8 y 16) no son metadatos accesorios: sin ellos, el router no podría aplicar el filtro correcto, la respuesta no podría citar su origen, y el sistema no podría garantizar que solo se redistribuye contenido con licencia compatible. El diseño del esquema de payload resultó, en la práctica, tan relevante para el resultado final como la elección del modelo de embeddings.

## 20.6 El grounding no es una consecuencia automática del retrieval

Recuperar los documentos correctos no garantiza, por sí solo, una respuesta fiable: sin restricciones explícitas en el prompt de generación, un LLM puede complementar el contexto recuperado con conocimiento propio, mezclando evidencia verificada con contenido no verificable. El grounding (sección 6.4) exige imponer estas restricciones de forma explícita — no inventar datos, citar cada afirmación por su fuente, declarar cuándo la evidencia es insuficiente — como una capa de control independiente del retrieval, no como una propiedad que se obtiene gratuitamente por el hecho de proporcionar contexto al modelo.

## 20.7 El chunking condiciona directamente la capacidad discriminativa del embedding

El análisis de longitud de las secciones de PMC mostró que una estrategia de chunking uniforme no es viable: los fragmentos demasiado extensos superan el rango en el que bge-m3 mantiene capacidad discriminativa y diluyen la representación semántica, mientras que los fragmentos demasiado pequeños fragmentan artificialmente secciones que ya eran coherentes. Esta observación motivó el criterio de chunking adaptativo descrito en la sección 8, en el que el umbral de división depende de la longitud real de cada sección, no de un tamaño fijo definido a priori.

## 20.8 El retrieval semántico requiere filtrado estructurado, no solo similitud vectorial

La similitud vectorial resuelve la proximidad semántica entre una consulta y un documento, pero no resuelve restricciones estructurales como la fuente, la fecha o el identificador de un artículo concreto. La elección de Qdrant (sección 10) estuvo motivada precisamente por esta necesidad: el filtrado sobre metadatos durante el recorrido del índice, no como post-procesamiento, es lo que permite que el router acote el espacio de búsqueda antes de calcular similitud, en lugar de calcular similitud sobre toda la colección y filtrar después.

## 20.9 La actualización incremental forma parte del propio sistema RAG, no de su mantenimiento posterior

Un sistema RAG orientado a literatura científica no puede considerarse completo si solo resuelve la consulta: el corpus sobre el que opera cambia con el tiempo — se publican artículos, se revisan registros, se retractan estudios — y un índice que no refleja esos cambios degrada

silenciosamente la calidad de sus respuestas. La actualización incremental (sección 14) no es, por tanto, una funcionalidad adicional sobre un sistema RAG ya terminado, sino una extensión necesaria de su ciclo de vida — de ahí que la Parte II se trate como evolución del sistema y no como un conjunto de tareas independientes del núcleo.

## 21. Problemas encontrados y decisiones técnicas

A diferencia de los aprendizajes de la sección anterior, centrados en el comportamiento del propio sistema RAG, esta sección recoge problemas de infraestructura y operación encontrados durante la evolución del sistema (Parte II). Las decisiones sobre las tecnologías centrales del pipeline — Qdrant, bge-m3, FastAPI, LangChain — ya se desarrollaron con sus alternativas en la Parte I, junto a la pieza que describen.

### 21.1 Límites de cuota en Gemini y arquitectura multiproveedor

Problema: el sistema dependió inicialmente de un único proveedor LLM (Gemini, con la clave hardcodeda). En uso intensivo se alcanzaron límites de cuota (HTTP 429), lo que bloqueaba la clasificación y la generación sin ninguna alternativa disponible. Decisión: sustituir la dependencia directa de Gemini por la fábrica multiproveedor de la sección 12. Resultado: un límite de cuota de un proveedor concreto deja de ser un punto único de fallo.

### 21.2 Tamaño de las dependencias CUDA de torch

Problema: sentence-transformers arrastra torch de forma transitiva, y el wheel por defecto de PyPI en Linux instala el runtime completo de CUDA (~2 GB de librerías nvidia-\*), aunque el proyecto ejecuta los embeddings exclusivamente en CPU. Decisión: instalar explícitamente la variante CPU-only de torch en un paso propio del Dockerfile antes de instalar el resto de dependencias. Resultado: la imagen no descarga ni instala librerías CUDA que nunca se usan.

### 21.3 Bloqueo de FastAPI por procesamiento CPU-bound

Problema: el parseo de XML y el procesamiento de la actualización del corpus son trabajo CPU-bound en Python puro, que bloquearía el event loop de FastAPI incluso ejecutándose en un hilo. Alternativa descartada: una cola con Celery o RQ, desproporcionada para un proyecto de portfolio sin infraestructura de colas ya desplegada. Decisión: ProcessPoolExecutor de un único worker (sección 14). Resultado: la API sigue respondiendo mientras una actualización corre en paralelo, verificado con un test que ejecuta ambas cosas a la vez.

### 21.4 Reinicio del contenedor a mitad de una actualización

Problema: si el contenedor de la API se reinicia mientras un job de actualización está en curso, el fichero de estado queda marcado “running” de forma permanente, aunque el proceso que lo escribió ya no existe. Decisión: en el arranque de FastAPI, comprobar el estado persistido y, si indica “running”, marcarlo explícitamente como fallido. Resultado: el frontend siempre recibe un estado consistente al reiniciar.

### 21.5 Gestión de duplicados en la ingesta

Problema: el mismo artículo puede aparecer en múltiples consultas o ingestas sucesivas. Decisión: calcular el identificador de cada punto como un UUID determinista (uuid5) a partir de su document\_id, en lugar de un identificador aleatorio. Resultado: el upsert de Qdrant sobrescribe el mismo punto en lugar de duplicarlo. La protección se refuerza con el botón de Ingestar deshabilitado durante la petición y el lock+cooldown de /corpus/update.

## 22. Resultados

El proyecto culmina en un sistema RAG biomédico capaz de cubrir el ciclo completo de gestión del conocimiento científico: incorporación de nueva literatura, procesamiento documental, indexación semántica, recuperación adaptativa, generación de respuestas fundamentadas y mantenimiento incremental del corpus.

La solución permite consultar tanto resúmenes temáticos como artículos concretos y comparaciones entre publicaciones, adaptando automáticamente la estrategia de recuperación a la intención de la consulta. Todas las respuestas se generan exclusivamente a partir del contexto recuperado e incorporan las fuentes que las respaldan, manteniendo la trazabilidad hasta el fragmento concreto utilizado como evidencia.

Desde el punto de vista de la arquitectura, el sistema desacopla la recuperación semántica del proveedor de inferencia, permitiendo utilizar indistintamente OpenAI, Gemini u Ollama sin modificar el pipeline RAG. Asimismo, incorpora mecanismos para ampliar y mantener el corpus de forma incremental sin interrumpir el servicio de consulta.

La solución se distribuye completamente contenerizada mediante Docker Compose, dispone de una interfaz web para todas las operaciones principales y cuenta con una batería de pruebas automatizadas que validan tanto los componentes individuales como el funcionamiento integral del sistema.

Más allá de implementar un sistema capaz de responder preguntas sobre literatura biomédica, **el proyecto demuestra cómo diseñar una arquitectura RAG mantenible y extensible**, donde la recuperación documental, la generación de respuestas y la gestión del corpus permanecen desacopladas y pueden evolucionar de forma independiente.

## 23. Competencias demostradas

1. Diseño de arquitecturas RAG con retrieval condicionado por la intención de la consulta, no por una única estrategia de búsqueda fija.
2. Evaluación comparativa de tecnologías centrales — base de datos vectorial, modelo de embeddings, framework de API, librería de orquestación — con alternativas descartadas argumentadas, no elegidas por defecto.
3. Migración de una integración externa deprecada a la infraestructura vigente sin interrumpir el servicio.
4. Diseño de una arquitectura multiproveedor de LLM mediante el patrón de fábrica, desacoplando la lógica de negocio del proveedor de inferencia concreto.
5. Diseño de procesamiento asíncrono para trabajo CPU-bound dentro de una API síncrona, con persistencia de estado y recuperación ante fallos.
6. Diseño de esquemas de datos para bases vectoriales con filtrado híbrido semántico y de metadatos, incluida la gestión de licencias de reutilización.
7. Diseño de prompts con grounding explícito para mitigar alucinaciones en un dominio de alta exigencia factual.

## 24. Limitaciones y trabajo futuro

### 24.1 Limitaciones metodológicas

- La evaluación del sistema se ha realizado mediante pruebas funcionales, pero no mediante un benchmark cuantitativo con métricas de recuperación como Recall@K, MRR o nDCG.
- La comparación entre modelos de embeddings se realizó sobre un corpus reducido y curado, por lo que las conclusiones obtenidas no pueden extrapolarse directamente a colecciones de gran escala.

## 24.2 Limitaciones arquitectónicas

- Ausencia de reranking de segunda etapa: bge-m3 es el modelo más robusto para la recuperación inicial, pero no necesariamente el más preciso para puntuar la relevancia final, especialmente en consultas transversales, donde compiten chunks de distintos papers.
- La ingesta bajo demanda (/ingest) permanece síncrona, manteniendo la conexión HTTP abierta durante todo el proceso.
- El worker de actualización incremental utiliza el contexto fork de multiprocessing, un riesgo teórico documentado pero no eliminado (sección 14.3).
- Ausencia de autenticación y de un modelo de despliegue multiusuario; el lock y el cooldown de /corpus/update mitigan el abuso básico, pero no sustituyen un control de acceso real.

## 24.3 Trabajo futuro

### Mejora de la recuperación

- Evaluación cuantitativa del retrieval mediante un conjunto de preguntas de referencia y métricas como Recall@K, Precision@K, MRR y nDCG.
- Incorporación de un reranker de segunda etapa para mejorar la precisión de los documentos entregados al modelo generativo.
- Recuperación híbrida combinando búsqueda vectorial y búsqueda léxica (BM25).

### Evolución del corpus

- Ampliación progresiva del corpus biomédico incorporando nuevas fuentes además de PubMed y PubMed Central.
- Incorporación de mecanismos para valorar automáticamente el nivel de evidencia científica de cada publicación.
- Actualización continua del índice mediante procesos programados.

### Evolución de la arquitectura

- Despliegue cloud.
- Sistema de autenticación.
- Cola distribuida.
- Observabilidad.
- Escalado horizontal.

### Experiencia de usuario

- Historial de consultas.
- Streaming de respuestas.

- Filtros de búsqueda.
- Exportación de resultados.

### Evolución del propio pipeline RAG

- Query rewriting.
- HyDE (Hypothetical Document Embeddings).
- Multi-query retrieval.
- Contextual compression.
- Citation verification.
- Retrieval adaptativo basado en confianza.
- Agentes especializados.
- Reranking cruzado.

La arquitectura desarrollada constituye una base suficientemente desacoplada para incorporar estas mejoras de forma incremental. La separación entre recuperación, generación, mantenimiento del corpus y proveedor LLM permite evolucionar cada componente de manera independiente sin alterar el funcionamiento global del sistema.

## 25. Conclusiones

El desarrollo de este proyecto ha permitido abordar el diseño de un sistema RAG desde una perspectiva integral, cubriendo todas las etapas del ciclo de vida de la información: adquisición del conocimiento, procesamiento documental, representación semántica, recuperación adaptativa, generación fundamentada de respuestas y mantenimiento del corpus a lo largo del tiempo.

Más allá de la implementación de una arquitectura concreta, el principal aprendizaje ha consistido en comprobar que **el rendimiento de un sistema RAG depende del equilibrio entre todos sus componentes**. La calidad de las respuestas no viene determinada únicamente por el modelo de lenguaje empleado, sino por decisiones previas como la selección de las fuentes documentales, la estrategia de chunking, el modelo de embeddings, el diseño del retrieval, la estructura de los metadatos o la forma en que se construye el contexto entregado al modelo generativo. El LLM resulta, en la práctica, el último eslabón de esa cadena — el que menos margen tiene para compensar un error cometido antes.

El proyecto también ha puesto de manifiesto la importancia de diseñar arquitecturas desacopladas y evolutivas. La incorporación de nuevos proveedores de LLM, la actualización incremental del corpus, la migración a los nuevos servicios de NCBI o la gestión asíncrona del mantenimiento del índice pudieron implementarse sin modificar el núcleo del pipeline RAG, precisamente porque la arquitectura fue concebida desde el principio como un conjunto de componentes independientes.

Como resultado, la solución desarrollada trasciende la demostración de un sistema capaz de responder preguntas sobre literatura biomédica. Constituye una **plataforma mantenible, reproducible y extensible**, en la que cada decisión técnica responde a un requisito concreto y cada componente puede evolucionar sin comprometer el funcionamiento del resto del sistema.

El desarrollo del proyecto ha puesto de manifiesto que construir un sistema RAG implica mucho más que integrar un modelo de lenguaje con una base de datos vectorial. Requiere

comprender cómo interactúan la recuperación semántica, la calidad de los datos, la estructura documental y la generación condicionada por contexto, así como asumir que pequeñas decisiones de diseño en cualquiera de estas etapas tienen un impacto directo sobre la calidad final de las respuestas.

*El verdadero reto de un sistema RAG no consiste en integrar un modelo de lenguaje, sino en diseñar correctamente el pipeline que determina qué conocimiento recupera, cómo lo representa y con qué evidencia fundamenta cada respuesta.*